



NICO ROBIN

Group Management Bot

Complete Technical Documentation • v1.0

"Archaeology of Power — Every group has a history. Robin reads it all."

Field	Value
Bot Identity	Nico Robin — Demon Child of the Straw Hats
Platform	Telegram (Bot API v7+)
Primary Language	Python 3.11+
Framework	python-telegram-bot v21 (asyncio)
Database	PostgreSQL + Redis
Version	1.0.0 May 2026

1. Project Overview

Nico Robin Bot is a production-grade Telegram group management bot themed after Nico Robin — archaeologist, Devil Fruit user, and the soul of the Straw Hat Pirates. The bot brings the same intellectual mastery Robin applies to Poneglyphs to your Telegram groups: reading every event, archiving every action, and wielding authority with quiet precision.

1.1 Design Philosophy

- Demon-Child Precision — every command responds in < 200 ms under normal load
- Archaeologist's Memory — full audit log of all moderation actions, forever
- Hana Hana no Mi — the bot can "bloom" handlers anywhere; modular plugin architecture
- Language of the Ancients — rich multilingual i18n with per-group locale
- Ohara Library — exhaustive inline /help with Robin-flavoured copy

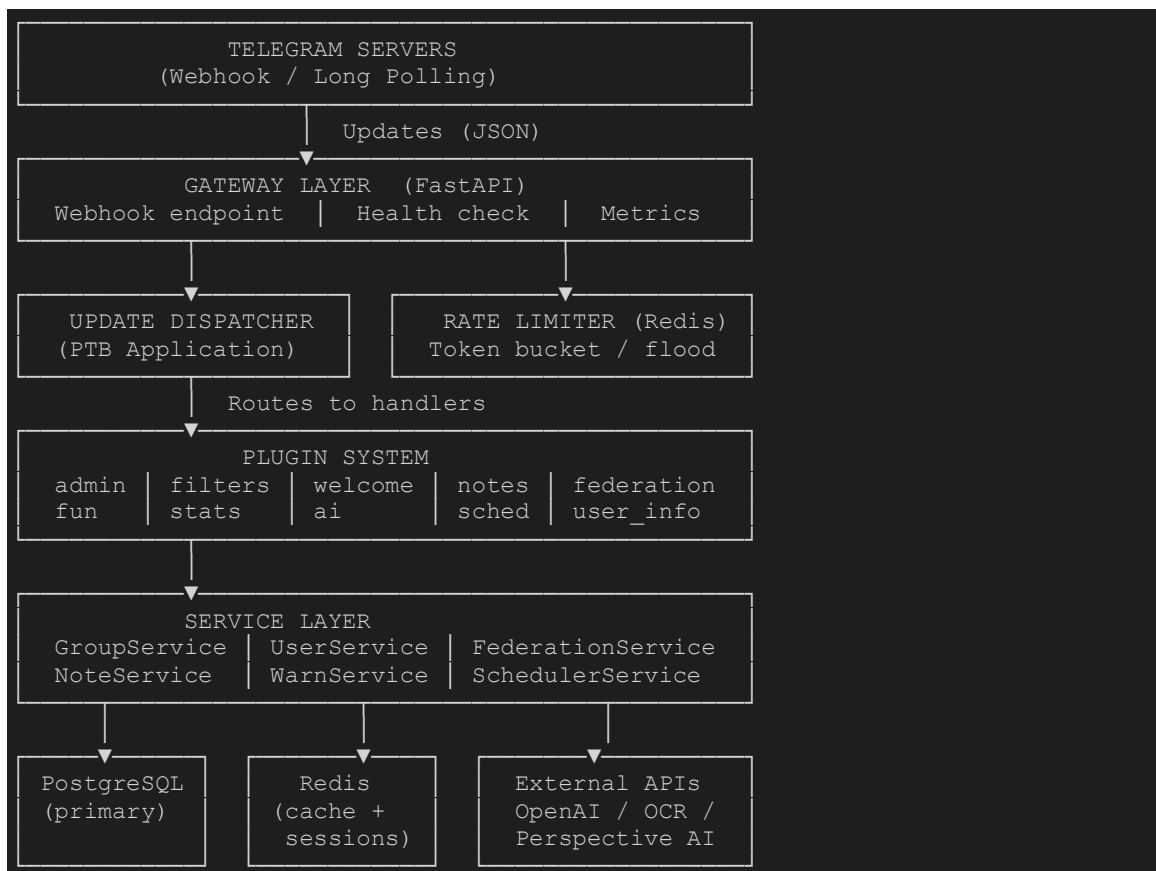
1.2 Feature Surface (at a glance)

Category	Capabilities
Administration	Ban, kick, mute, warn, lock, pin, purge, slow mode, link control
Anti-Spam / Filters	Flood control, regex filters, bad-words, anti-raid, CAPTCHA
Welcome System	Custom welcome/farewell, buttons, media, per-user timeout
Notes & Tags	Saveable notes with Markdown, buttons, and media attachments
User Tracking	Join/leave logs, warn records, activity stats, reputation
Scheduler	Timed mutes, timed bans, announcement scheduling
Federated Bans	Cross-group ban propagation via federation system
Fun & Engagement	Polls, quizzes, word games, Robin quotes
Analytics	Group activity graphs, top members, message heatmaps
AI Integration	GPT-powered moderation suggestions, toxic-text detection

2. System Architecture

2.1 High-Level Diagram

The bot follows a layered, event-driven architecture:



2.2 Technology Stack

Component	Technology	Role	Status
Runtime	Python 3.11	Async throughout	Required
Bot Framework	python-telegram-bot 21	Native asyncio, JobQueue, Webhooks	Required
Web Gateway	FastAPI + Uvicorn	Webhook receiver, health, metrics	Required
Primary DB	PostgreSQL 15	Groups, users, warns, notes, logs	Required
ORM	SQLAlchemy 2 (async)	Async sessions, Alembic migrations	Required
Cache / Queue	Redis 7	Rate limits, sessions, task queue	Required

Component	Technology	Role	Status
Task Queue	Celery + Redis	Scheduled bans/mutes, announcements	Required
AI Layer	OpenAI GPT-4o-mini	Moderation assist, toxic detection	Optional
Monitoring	Prometheus + Grafana	Latency, error rate, command usage	Optional
Containerise	Docker + Compose	One-command deployment	Recommended

2.3 Database Schema (ERD — key tables)

<u>groups</u>	<u>users</u>	<u>warns</u>	
group_id (PK)	user_id (PK)	warn_id (PK)	
title	username	group_id (FK)	
locale	first_name	user_id (FK)	
prefix	last_name	reason	
welcome_text	reputation	admin_id (FK)	
rules	join_date	issued_at	
max_warns	ban_count	expires_at	
mute_time	warn_count		
captcha_on	is_bot		
		<u>notes</u>	<u>filters</u>
<u>group_members</u>	<u>federations</u>	note_id (PK)	filter_id (PK)
group_id (FK)	fed_id (PK)	group_id (FK)	group_id (FK)
user_id (FK)	name	name	trigger
joined_at	owner_id	content	action
is_admin	created_at	media_id	regex
permissions		button_json	created_at
		created_by	

3. Project Directory Structure

```
nico_robin_bot/
├── main.py                # Entry point — builds app & runs
├── config.py              # Settings via pydantic-settings
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── alembic/               # DB migrations
│   ├── env.py
│   └── versions/
├── bot/
│   ├── app.py             # PTB Application factory
│   ├── dispatcher.py      # Handler registration
│   ├── middleware/
│   │   ├── rate_limiter.py
│   │   ├── flood_control.py
│   │   └── auth.py        # Permission checks
│   ├── plugins/           # One file = one feature domain
│   │   ├── admin.py       # ban/kick/mute/warn/pin/lock
│   │   ├── filters.py     # Regex & keyword filters
│   │   ├── welcome.py     # Join/leave messages
│   │   ├── notes.py       # Saveable notes & tags
│   │   ├── purge.py       # Message bulk-delete
│   │   ├── captcha.py     # New-member CAPTCHA
│   │   ├── federation.py  # Cross-group ban sync
│   │   ├── scheduler.py   # Timed actions
│   │   ├── stats.py       # Analytics
│   │   ├── fun.py         # Polls, games, quotes
│   │   ├── ai_mod.py      # AI moderation assist
│   │   ├── user_info.py   # /whois, /id, /info
│   │   └── settings.py    # Per-group config UI
├── services/
│   ├── group_service.py
│   ├── user_service.py
│   ├── warn_service.py
│   ├── note_service.py
│   ├── filter_service.py
│   ├── federation_service.py
│   └── scheduler_service.py
├── models/                # SQLAlchemy ORM models
│   ├── base.py
│   ├── group.py
│   ├── user.py
│   ├── warn.py
│   ├── note.py
│   ├── filter.py
│   └── federation.py
├── utils/
│   ├── permissions.py     # Hierarchy checks
│   ├── decorators.py      # @admin_only, @group_only, etc.
│   ├── formatters.py      # Message templates
│   ├── keyboards.py       # InlineKeyboard builders
│   └── robin_quotes.py    # Flavour text
├── gateway/
│   └── webhook.py         # FastAPI webhook receiver
├── tasks/                 # Celery tasks
│   ├── celery_app.py
│   ├── ban_tasks.py
│   └── announce_tasks.py
├── i18n/
└── en.json
```

```
└─ hi.json  
└─ ja.json
```

4. Permission & Role System

Robin respects hierarchy — just as she navigated the World Government, Marines, and the Straw Hats. The bot implements a five-tier system:

Level	Who	Capabilities	Source
0 — Sudo Owner	Bot developer(s)	Unrestricted — can act in any group	SUDO_USERS env list
1 — Bot Owner	Bot added by this user	Full group control, add admins	groups.owner_id
2 — Admin	Telegram group admin	Ban, mute, warn, filters, notes	Telegram API
3 — Moderator	Bot-level mod role	Warn, mute, filter mgmt	group_members.permissions
4 — User	Regular members	Help, info, fun commands only	Default
5 — Restricted	Muted / warned members	Read-only; no commands	group_members.is_restricted

4.1 Permission Decorator

```
# utils/decorators.py
def admin_only(func):
    @wraps(func)
    async def wrapper(update: Update, context: ContextTypes.DEFAULT_TYPE):
        user = update.effective_user
        chat = update.effective_chat
        if user.id in settings.SUDO_USERS:
            return await func(update, context)
        member = await context.bot.get_chat_member(chat.id, user.id)
        if member.status not in (ChatMember.ADMINISTRATOR, ChatMember.OWNER):
            await update.message.reply_text(
                '🌸 "Interesting... but you lack the authority." — Robin'
            )
            return
        return await func(update, context)
    return wrapper
```

5. Full Command Reference

5.1 Administration Commands

Command	Description
<code>/ban [reply @user] [reason]</code>	Permanently ban a user. Logs reason to DB.
<code>/unban @user</code>	Unban a previously banned user.
<code>/kick [reply @user] [reason]</code>	Remove user without ban (can rejoin).
<code>/mute [reply @user] [time]</code>	Mute user. Time: 1m, 2h, 3d. Permanent if omitted.
<code>/unmute [reply @user]</code>	Restore muted user's speak permission.
<code>/tban [reply @user] [time]</code>	Temporary ban with auto-unban via scheduler.
<code>/tmute [reply @user] [time]</code>	Temporary mute with auto-unmute.
<code>/warn [reply @user] [reason]</code>	Issue a warning. Auto-action at threshold.
<code>/unwarn [reply @user]</code>	Remove the most recent warning.
<code>/resetwarn [reply @user]</code>	Clear all warnings for a user.
<code>/warns [reply @user]</code>	List all active warnings for a user.
<code>/promote [reply @user] [title]</code>	Promote user to admin with optional title.
<code>/demote [reply @user]</code>	Demote admin to regular member.
<code>/pin [reply] [loud silent]</code>	Pin a message. Silent = no notification.
<code>/unpin</code>	Unpin the current pinned message.
<code>/unpinall</code>	Unpin all pinned messages (owner only).
<code>/purge [N]</code>	Delete last N messages (reply = from that msg).
<code>/del</code>	Delete the replied-to message.
<code>/title [reply @user] [text]</code>	Set a custom admin title.
<code>/setdesc [text]</code>	Set the group description.
<code>/setpic [reply to photo]</code>	Set the group profile picture.

5.2 Restrictions & Locks

Command	Description
<code>/lock [type]</code>	Lock a content type. Types: text, photo, video, sticker, gif, link, forward, game, inline, all
<code>/unlock [type]</code>	Unlock a previously locked content type.
<code>/locks</code>	Show all current lock statuses in this group.
<code>/slowmode [secs]</code>	Set slow mode delay (0 to disable).
<code>/antispam on off</code>	Toggle built-in anti-spam engine.

Command	Description
<code>/antiraid on off</code>	Toggle raid protection (auto-ban rapid joins).
<code>/antilink on off</code>	Toggle link-sharing restriction.
<code>/captcha on off</code>	Toggle CAPTCHA for new members.

5.3 Welcome & Farewell

Command	Description
<code>/setwelcome [text]</code>	Set a custom welcome message. Supports {first}, {username}, {chat}, {count} placeholders.
<code>/resetwelcome</code>	Restore the default welcome message.
<code>/welcome on off</code>	Enable or disable welcome messages.
<code>/setfarewell [text]</code>	Set a custom goodbye message.
<code>/farewell on off</code>	Enable or disable farewell messages.
<code>/cleanwelcome on off</code>	Auto-delete the previous welcome message when a new member joins.
<code>/setrules [text]</code>	Set group rules shown via <code>/rules</code> .
<code>/rules</code>	Display the current group rules.
<code>/welcometest</code>	Preview the welcome message as if you just joined.

5.4 Notes & Tags

Command	Description
<code>/save [name] [content]</code>	Save a note. Can include text, media (reply), and buttons.
<code>/get [name] or #name</code>	Retrieve a saved note.
<code>/notes</code>	List all saved notes in this group.
<code>/clear [name]</code>	Delete a note.
<code>/clearall</code>	Delete all notes (admin only).
<code>/privatenotes on off</code>	Send notes via DM instead of in-group.

5.5 Filters

Command	Description
<code>/filter [trigger] [reply]</code>	Add a keyword/regex filter. When triggered: reply, delete, warn, or ban.
<code>/stop [trigger]</code>	Remove a filter.
<code>/filters</code>	List all active filters.
<code>/filteraction [action]</code>	Default filter action: reply delete warn ban mute.

5.6 Flood Control

Command	Description
<code>/setflood [N]</code>	Set max messages per 5-second window (0 = off).
<code>/setfloodmode [mode]</code>	Action on flood: ban kick mute tban [time] tmute [time].
<code>/flood</code>	Show current flood settings.

5.7 Federation Commands

Command	Description
<code>/newfed [name]</code>	Create a new federation (bot owner only).
<code>/joinfed [fed_id]</code>	Join your group to a federation.
<code>/leavefed</code>	Leave the current federation.
<code>/fban @user [reason]</code>	Ban a user across all groups in the federation.
<code>/funban @user</code>	Lift a federation ban.
<code>/fedinfo [fed_id]</code>	Show federation metadata and member groups.
<code>/fedadmins</code>	List admins of the current federation.
<code>/fpromote @user</code>	Promote a user as federation admin.
<code>/fdemote @user</code>	Demote a federation admin.

5.8 User Information

Command	Description
<code>/id [reply @user]</code>	Show Telegram ID of user or current chat.
<code>/whois [reply @user]</code>	Full profile: ID, username, warns, join date, reputation, fed-bans.
<code>/info [reply @user]</code>	Short summary card with inline buttons.
<code>/admins</code>	List all admins in the group.
<code>/staff</code>	List bot-level moderators.
<code>/mute_list</code>	List currently muted members.
<code>/ban_list</code>	List banned members tracked by bot.

5.9 Scheduler & Announcements

Command	Description
<code>/schedule [time] [text]</code>	Schedule an announcement. Time: ISO8601 or '+2h30m'.
<code>/listschedules</code>	Show all scheduled tasks for this group.

Command	Description
<code>/cancelschedule [id]</code>	Cancel a scheduled task by ID.
<code>/settz [timezone]</code>	Set group timezone (e.g. Asia/Kolkata).

5.10 Statistics & Analytics

Command	Description
<code>/stats</code>	Group statistics: member count, messages today, warns issued, bans today.
<code>/topusers</code>	Top 10 most active users by message count.
<code>/activity</code>	Message activity graph for the past 7 days.
<code>/chatreport</code>	Full weekly report as a formatted document.
<code>/mystats</code>	Personal stats for the calling user.

5.11 Fun & Engagement

Command	Description
<code>/poll [question]</code>	Create a Telegram native poll.
<code>/quiz [question] [ans...]</code>	Create a quiz poll with correct answer.
<code>/robin</code>	Random Nico Robin quote.
<code>/devil [fruit_name]</code>	Describe a One Piece Devil Fruit (fun trivia).
<code>/roll [NdM]</code>	Roll dice. e.g. /roll 2d6.
<code>/flip</code>	Flip a coin.
<code>/8ball [question]</code>	Magic 8-ball answer.
<code>/riddle</code>	Post a riddle; first correct reply wins.

5.12 Bot Configuration (Owner / Sudo)

Command	Description
<code>/setprefix [char]</code>	Change the command prefix (default /).
<code>/setlocale [code]</code>	Set group language: en hi ja es ru.
<code>/setwarnlimit [N]</code>	Max warnings before auto-action.
<code>/setwarnaction [action]</code>	Action at warn limit: ban kick mute tban [t].
<code>/setmuteaction [action]</code>	Default mute duration.
<code>/toggleai on off</code>	Enable/disable AI moderation assist.
<code>/blacklistword [word]</code>	Add a word to the global bad-word list.
<code>/blacklist</code>	Show bad-word list.

Command	Description
<code>/logs</code>	Toggle action logging to a designated log channel.
<code>/setlogchannel</code>	Set the log channel (forward any message from it).
<code>/broadcast [text]</code>	Broadcast to all groups (sudo only).
<code>/ping</code>	Check bot latency.
<code>/reload</code>	Hot-reload plugin handlers without restart.
<code>/uptime</code>	Show bot uptime and system stats.

6. Anti-Spam & Anti-Raid Engine

6.1 Flood Detection Algorithm

The flood engine uses a token-bucket per (group_id, user_id) stored in Redis with a TTL-based sliding window:

```
# middleware/flood_control.py
FLOOD_KEY = 'flood:{chat_id}:{user_id}'

async def check_flood(chat_id: int, user_id: int, limit: int) -> bool:
    key = FLOOD_KEY.format(chat_id=chat_id, user_id=user_id)
    pipe = redis.pipeline()
    now = time.time()
    pipe.zadd(key, {now: now})
    pipe.zremrangebyscore(key, 0, now - 5)    # 5-second window
    pipe.zcard(key)
    pipe.expire(key, 10)
    _, _, count, _ = await pipe.execute()
    return count > limit
```

6.2 Anti-Raid Protection

Raid mode activates automatically when more than N users join within T seconds. While active, new members are immediately restricted until an admin clears the raid flag.

```
RAID_THRESHOLD = 10    # joins
RAID_WINDOW    = 30    # seconds
RAID_KEY       = 'raid:{chat_id}'

async def on_member_join(update, context):
    chat_id = update.effective_chat.id
    count = await redis.incr(RAID_KEY.format(chat_id=chat_id))
    await redis.expire(RAID_KEY.format(chat_id=chat_id), RAID_WINDOW)
    if count >= RAID_THRESHOLD:
        await activate_raid_mode(chat_id, context)
```

6.3 CAPTCHA System

When a new member joins a CAPTCHA-enabled group, Robin's bot immediately restricts them and sends a button challenge. Failure to solve within 90 seconds results in an automatic kick.

- Math CAPTCHA — simple arithmetic challenge
- Button CAPTCHA — tap the correct emoji from 4 options
- Text CAPTCHA — type a displayed string

7. AI Moderation Layer

The optional AI moderation module augments rule-based filters with semantic understanding — identifying toxicity, NSFW text, doxxing attempts, and context-sensitive spam that regex cannot catch.

7.1 AI Pipeline

```
# bot/plugins/ai_mod.py
async def ai_moderate(message_text: str, context: dict) -> ModerationResult:
    response = await openai.chat.completions.create(
        model='gpt-4o-mini',
        messages=[
            {'role': 'system', 'content': MODERATION_SYSTEM_PROMPT},
            {'role': 'user', 'content': message_text},
        ],
        response_format={'type': 'json_object'},
        max_tokens=150,
    )
    result = json.loads(response.choices[0].message.content)
    # result: {score: 0-1, category: str, action: 'none'|'warn'|'delete'|'ban'}
    return ModerationResult(**result)
```

7.2 Moderation Categories

Category	Threshold	Default Action	Notes
hate_speech	> 0.85	delete + warn	Direct ethnic/racial slurs
harassment	> 0.80	warn	Targeted personal attacks
nsfw_text	> 0.75	delete	Sexually explicit language
doxxing	> 0.70	delete + ban	Personal info of others
spam_promo	> 0.65	delete	Unsolicited advertisement
self_harm	> 0.60	notify_admin	Content encouraging self-harm

8. Configuration Reference

8.1 Environment Variables (.env)

```
# — Core —————
BOT_TOKEN=7xxxxxxx:AAxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
WEBHOOK_URL=https://yourdomain.com/webhook
WEBHOOK_SECRET=your_random_secret
PORT=8000

# — Sudo Users —————
SUDO_USERS=123456789,987654321

# — PostgreSQL —————
DATABASE_URL=postgresql+asyncpg://robin:password@localhost:5432/robin_db

# — Redis —————
REDIS_URL=redis://localhost:6379/0

# — Celery —————
CELERY_BROKER_URL=redis://localhost:6379/1
CELERY_RESULT_BACKEND=redis://localhost:6379/2

# — Optional: AI —————
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
AI_MODERATION_ENABLED=true
AI_SCORE_THRESHOLD=0.75

# — Optional: Log Channel —————
LOG_CHANNEL_ID=-100xxxxxxxxxx

# — Bot Personality —————
BOT_NAME=Nico Robin
DEFAULT_LOCALE=en
DEFAULT_PREFIX=/
```

8.2 Per-Group Settings (stored in DB)

Key	Type	Default	Description
welcome_enabled	bool	true	Send welcome messages
farewell_enabled	bool	true	Send farewell messages
captcha_enabled	bool	false	CAPTCHA for new members
antispam_enabled	bool	true	Flood detection
antiraid_enabled	bool	false	Raid auto-restrict
ai_mod_enabled	bool	false	AI moderation
max_warns	int	3	Warns before auto-action
warn_action	string	ban	Action at warn limit
flood_limit	int	5	Messages per 5 sec

Key	Type	Default	Description
flood_action	string	mute	Action on flood
mute_time	int	3600	Default mute seconds
locale	string	en	Language code
prefix	string	/	Command prefix
log_channel_id	bigint	null	Audit log destination

9. Deployment Guide

9.1 Docker Compose (Recommended)

```
# docker-compose.yml
version: '3.9'
services:
  bot:
    build: .
    restart: unless-stopped
    env_file: .env
    depends_on: [db, redis]
    ports: ['8000:8000']

  celery_worker:
    build: .
    command: celery -A tasks.celery_app worker --loglevel=info
    env_file: .env
    depends_on: [redis]

  celery_beat:
    build: .
    command: celery -A tasks.celery_app beat --loglevel=info
    env_file: .env
    depends_on: [redis]

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: robin
      POSTGRES_PASSWORD: password
      POSTGRES_DB: robin_db
    volumes: ['pgdata:/var/lib/postgresql/data']

  redis:
    image: redis:7-alpine
    volumes: ['redisdata:/data']

volumes:
  pgdata:
  redisdata:
```

9.2 Startup Sequence

1. Clone the repo and copy .env.example to .env — fill all required values
2. docker compose up -d db redis — start infrastructure
3. alembic upgrade head — apply all database migrations
4. docker compose up -d — start bot, celery worker, and celery beat
5. Set the Telegram webhook: GET
`https://api.telegram.org/bot{TOKEN}/setWebhook?url={WEBHOOK_URL}`
6. Verify: send /ping to the bot in any group

9.3 Performance Tuning

- Run Uvicorn with `--workers 2` on a dual-core VPS; single-worker is fine for < 50 groups
- Use connection pooling: `pool_size=20`, `max_overflow=10` in SQLAlchemy async engine
- Redis pipeline all flood-check ops to reduce round trips
- Cache group settings in Redis with a 60-second TTL to avoid repeated DB hits
- Use PTB's JobQueue for lightweight repeating tasks instead of Celery for sub-minute intervals

10. Core Code Patterns

10.1 main.py — Application Bootstrap

```
import asyncio
from telegram.ext import Application
from bot.dispatch import register_all_handlers
from gateway.webhook import create_app
from config import settings
import uvicorn

async def main():
    ptb_app = (
        Application.builder()
            .token(settings.BOT_TOKEN)
            .build()
    )
    register_all_handlers(ptb_app)

    web_app = create_app(ptb_app)  # FastAPI wrapper

    config = uvicorn.Config(
        web_app, host='0.0.0.0', port=settings.PORT,
        log_level='info', loop='asyncio'
    )
    server = uvicorn.Server(config)

    async with ptb_app:
        await ptb_app.bot.set_webhook(
            url=f'{settings.WEBHOOK_URL}/webhook',
            secret_token=settings.WEBHOOK_SECRET,
        )
        await ptb_app.start()
        await server.serve()
        await ptb_app.stop()

if __name__ == '__main__':
    asyncio.run(main())
```

10.2 Sample Plugin — admin.py (Ban Handler)

```
from telegram import Update
from telegram.ext import CommandHandler, ContextTypes
from utils.decorators import admin_only, group_only
from services.user_service import UserService
from utils.formatters import ban_message

@group_only
@admin_only
async def ban(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg = update.effective_message
    target = await resolve_target(update, context)  # reply or @mention
    if not target:
        return await msg.reply_text('🌸 Who should I ban? Reply to a message or tag someone.')

    reason = ' '.join(context.args[1:]) if context.args else 'No reason provided'
```

```
chat_id = update.effective_chat.id

await context.bot.ban_chat_member(chat_id, target.id)
await UserService.log_ban(chat_id, target.id, update.effective_user.id,
reason)

await msg.reply_text(ban_message(target, reason))
await log_action(context, chat_id, 'ban', target, reason)

def register(app):
    app.add_handler(CommandHandler('ban', ban))
```

11. Logging & Audit Trail

Every moderation action is stored in an `action_logs` table and optionally forwarded to a designated Telegram log channel. The log channel messages are formatted in Robin's voice.

```
# Example log channel message
🌸 [BAN]
Admin: @johansen_droid (ID: 123456789)
Target: @spammer_123 (ID: 987654321)
Group: The Thousand Sunny Fan Club
Reason: Advertising crypto scams
Time: 2026-05-11 14:32:01 UTC

— Nico Robin has added another Poneglyph to the record.
```

Table	Key Columns	Purpose
<code>action_logs</code>	<code>group_id</code> , <code>actor_id</code> , <code>target_id</code> , <code>action</code> , <code>reason</code> , <code>created_at</code>	All mod actions
<code>message_logs</code>	<code>group_id</code> , <code>user_id</code> , <code>message_id</code> , <code>content_hash</code> , <code>deleted_at</code>	Deleted/edited messages
<code>join_logs</code>	<code>group_id</code> , <code>user_id</code> , <code>joined_at</code> , <code>invite_link</code> , <code>captcha_passed</code>	Member join events
<code>warn_log</code>	<code>group_id</code> , <code>target_id</code> , <code>admin_id</code> , <code>reason</code> , <code>auto_action</code> , <code>created_at</code>	Warning history
<code>fed_ban_log</code>	<code>fed_id</code> , <code>target_id</code> , <code>admin_id</code> , <code>reason</code> , <code>created_at</code>	Federation bans

12. Internationalisation (i18n)

12.1 Locale Structure

```
// i18n/en.json (excerpt)
{
  'ban.success': '🌸 {target} has been banned. Even the sea of knowledge has shores.',
  'ban.no_target': '🌸 "Interesting... but who should I restrain?" – Robin',
  'mute.success': '🌸 {target} has been silenced for {duration}.',
  'warn.issued': '🌸 Warning {count}/{max} issued to {target}. Reason: {reason}',
  'warn.limit_reached': '🌸 {target} has reached the warning limit. Taking action: {action}.',
  'captcha.prompt': '🌸 Welcome to {chat}! Please solve this to prove you are human:',
  'captcha.passed': '🌸 Verified. Welcome aboard, {user} – the library is open to you.',
  'captcha.failed': '🌸 You could not pass the test. The sea does not wait.',
}
```

12.2 Supported Locales

Code	Language	Status
en	English	Complete
hi	Hindi	Complete
ja	Japanese	Complete
es	Spanish	In progress
ru	Russian	In progress
pt	Portuguese	Planned
ar	Arabic	Planned

13. requirements.txt

```
# Core
python-telegram-bot[webhooks,job-queue]==21.5
fastapi==0.111.0
uvicorn[standard]==0.29.0

# Database
sqlalchemy[asyncio]==2.0.30
asyncpg==0.29.0
alembic==1.13.1

# Cache / Queue
redis[hiredis]==5.0.4
celery==5.4.0

# Config
pydantic-settings==2.2.1
python-dotenv==1.0.1

# AI (optional)
openai==1.30.1

# Utilities
httpx==0.27.0
Pillow==10.3.0
aiofiles==23.2.1
babel==2.15.0

# Monitoring (optional)
prometheus-client==0.20.0
structlog==24.2.0
```

14. Development Roadmap

Version	Features	Status
v1.0	Core admin commands, warns, notes, filters, welcome, flood, logs	Complete
v1.1	Federation system, CAPTCHA, anti-raid, scheduler, timed bans	Complete
v1.2	AI moderation layer, analytics dashboard, /chatreport	In Progress
v1.3	Multi-language support (7 locales), custom command aliases	Planned
v1.4	Web dashboard (FastAPI + React) for group admins	Planned
v1.5	Reputation system, leaderboards, economy mini-games	Planned
v2.0	Self-hosted federation server, group templates, Robin AI persona chat	Vision

15. Robin Personality — Quote Bank (Sample)

Every moderation response uses a rotating set of Robin-themed messages to keep the bot's personality consistent and memorable:

- Ban: "The sea of knowledge has limits for those who abuse it."
- Mute: "Silence can speak louder than any words."
- Warn: "Consider this a page in your personal Poneglyph."
- Kick: "Some doors close so others may open."
- Flood detected: "Too many words obscure the truth. Slow down."
- Raid detected: "An attack from all sides. Activating Hana Hana no Mi..."
- CAPTCHA fail: "The sea does not wait for those who hesitate."
- Note saved: "Archived. Even the Void Century deserves documentation."
- Welcome: "Every great crew began with a single member."
- Admin promote: "Authority is a privilege. Use it wisely."

 **Nico Robin Group Management Bot** 

"I want to live" — and so does every group under Robin's watch.

Technical Documentation v1.0 | Built by Johan (johan-droid) | May 2026